CSIT Project 2026: Answers to the Judging Criteria

Dyslexia handwriting screening with a convolutional neural network

Before submitting: items shown in **[orange brackets]** are placeholders for final results, figure numbers or details that were not available when this was drafted. The only measured figures so far are from epoch 1 of the default configuration (validation accuracy 0.724, macro F1 0.718), so these should not be quoted as final results. The text assumes the public dyslexia handwriting letter dataset with three classes (Normal, Reversal, Corrected); adjust the class names if yours differ.

Data Understanding and Exploration /15

Data Loading and Cleaning (5)

The dataset is made of images of single handwritten letters, so “missing data” means unreadable, empty or corrupted image files. We scanned every file, removed any that failed to open or were blank, and checked for exact duplicates across the train and test folders so the test score is not inflated.

Every image was converted to grayscale, resized to one fixed size, and its pixel values scaled to 0–1, because a CNN needs inputs of the same shape and scale. Unusual samples, such as very faint strokes or letters touching the edge, were kept rather than deleted, because real children’s handwriting looks like that and the model has to cope with it. We split off a validation set from the training data, stratified by class, so tuning decisions never touched the test set.

Exploratory Data Analysis (EDA) (5)

We counted images per class and found **[class balance, e.g. Reversal and Corrected are smaller than Normal]**. That imbalance is why we chose macro F1 over plain accuracy as our main metric.

We plotted sample grids for each class, which showed that Corrected letters often look very close to Normal ones. That explains most of the confusion we later saw between those two classes. We also looked at average pixel intensity and stroke thickness per class, and checked image sizes before resizing. **[insert class-count bar chart and sample grid figure numbers]**

Feature Engineering (5)

A CNN learns its own features from pixels, so our engineering effort went into the inputs and into augmentation. The key design choice: **we deliberately did not use horizontal flipping**, a standard augmentation, because mirroring a letter turns a normal “b” into a “d”. That would create reversals and label them as Normal, corrupting the exact signal we are trying to detect.

Instead we used small rotations, small shifts and slight scaling, which copy natural variation in children’s writing without changing the letter’s direction. We also treated augmentation as a hyperparameter (on/off) during tuning, so its effect is measured rather than assumed.

Data Modelling and Results /25

Model Selection: appropriateness and justification (5 + 5)

We used a convolutional neural network because the problem is image classification. CNNs detect local shapes like strokes, curves and orientation, which is exactly what separates a reversed letter from a normal one. Classical models on flattened pixels lose that spatial structure. **[if you have a baseline, e.g. logistic regression or random forest on pixels, name it and give its score]**

We kept the network small (base width 32 filters, dropout 0.3) on purpose. Our results have to be reproducible on an ordinary laptop CPU, and dropout reduces overfitting on a dataset where many letters look alike. Hyperparameters were chosen by a tuning run of **[9]** configurations over width, dropout, learning rate, batch size and augmentation, with the best chosen on validation macro F1.

Model Evaluation (3)

We report accuracy, precision, recall, F1 per class, macro F1, one-vs-rest ROC-AUC, and a confusion matrix. Macro F1 is our main metric because it weights each class equally, so the model cannot score well by just predicting the majority class.

We pay particular attention to **recall on the Reversal class**: in a screening setting, missing a child who may need support costs more than a false alarm that a teacher can check. Final test results: accuracy **[X]**, macro F1 **[X]**, Reversal recall **[X]**, ROC-AUC **[X]**.

Data Modelling: improving performance, flexibility and scalability (4)

Accuracy would improve most from more varied data: more writers, more ages, and whole words or sentences rather than isolated letters. We would split data by writer so the model is tested on children it has never seen. Technically, a GPU would allow longer training and transfer learning from a pretrained model.

For scalability, the trained model is small enough to run on a phone or a school laptop, and the pipeline is scripted end to end, so new data can be added and the model retrained without manual steps. For flexibility, the same approach could extend to other handwriting markers such as letter spacing or line alignment.

Results: communicating methodology (3)

Our pipeline runs in order: load and clean images → explore class balance and samples → augment without flipping → tune the CNN on a validation set → train the final model → evaluate once on the held-out test set.

Each finding links back to the data. For example, the class imbalance found in EDA drove the choice of macro F1, and the visual similarity between Corrected and Normal letters explains the main error in the confusion matrix. **[point to your confusion matrix figure]**

Soundness of approach and methodology (5)

The approach is sound because the test set is untouched until the end, tuning is logged, and every choice is justified by the data.

In short: we built a lightweight model that flags letter reversals in children's handwriting with **[X]** macro F1, trained in about an hour on a CPU. It is intended as an early screening aid for teachers, **not a diagnosis**, since dyslexia is assessed by professionals using much more than handwriting.

Documentation and Code /10

Code Quality and Documentation (10)

The code is split into separate modules for data loading, model definition, training, tuning and report generation, rather than one long script. Settings are kept in one config, and the best tuning config is saved to best_config.json so results can be reproduced.

Training writes a log per epoch (loss, accuracy, macro F1, learning rate, time), and tuning progress is saved after each run so it can resume after an interruption. Functions have docstrings, and comments explain non-obvious choices such as why flipping is disabled.
[mention your README, requirements.txt and random seed; add them if missing]